

Codes for inventory management system

Add product function

```
# a function that allow user to add product that store in product.txt file
def add_product():

    while True:
        #prompt user to enter product id that start with the format P001
        #Ensure product ID user enter starts with "P" and is followed by 3 digits
        productid = input("Enter Product ID (ex: P001): ")
        if len(productid) != 4 or not productid.startswith('P') or not productid[1:].isdigit():
            print("Invalid Product ID format. Please enter product ID starting with 'P' followed
by 3 digits (ex: P001).")
            continue

        # Check if the product ID user enter already exists in the product.txt file
        try:
            with open('product.txt', 'r') as productfile:
                lines = productfile.readlines()
        except IOError as e:
            print(e)
            print("Cannot read from file!")

        for line in lines:
            if line.startswith("Product ID: ") and productid in line:
                print("This Product ID already exists.") # Inform the user if the product ID
already exists

                # Prompt user to decide whether to try again
                while True:
                    user_choice = input("Do you want to enter product id again? (y/n): ")
                    if user_choice == 'y':
                        break # If user chooses 'y', prompt to enter product ID again
                    elif user_choice == 'n':
                        return # Exit the function if user chooses 'n'
                    else:
                        print("Please enter 'y' for yes or 'n' for no.") # If user enters invalid
input
                        break
                else:
                    #Break the loop if user enters a valid product ID format and no duplicate is found
                    break

        # Prompt user to enter other product details
        productname = input("Enter Product Name: ")
        productdes = input("Enter Product Description: ")

        #prompt user to enter product price
        while True:
            try:
                productprice = float(input("Enter Product Price: RM"))
                if productprice <=0:
```

```

        print("Please enter a valid product price.")
    else:
        break
except:
    print("Invalid product price. Please enter a valid product price.")

#prompt user to enter product quantity
while True:
    try:
        productquantity = int(input("Enter Product Quantity: "))
        if productquantity < 1:
            print("Please enter a positive integer for product quantity.")
        else:
            break #break the loop if the quantity user enter is valid
    except:
        print("Invalid input! Please enter a valid number for product quantity.")

# Store the new product information into the product.txt file
try:
    with open('product.txt', 'a') as productfile:
        productfile.write(f"Product ID: {productid}\n")
        productfile.write(f"Product Name: {productname}\n")
        productfile.write(f"Product Description: {productdes}\n")
        productfile.write(f"Product Price: RM{productprice}\n")
        productfile.write(f"Product Quantity: {productquantity}\n")
        productfile.write("\n") # Add a blank line to separate entries
except IOError as e:
    print(e)
    print('Cannot read from file!')

print("Product added successfully!")

```

Update product details function

```

#a function that allow user to update the product details of existing product
def update_product_details():
    while True:
        product_id_to_update = input("Enter product ID to update: ")

        # Check if the product ID format user enter is valid or not
        if len(product_id_to_update) != 4 or not product_id_to_update.startswith('P') or not
product_id_to_update[1:].isdigit():
            print("Invalid Product ID format. Please enter product ID starting with 'P' followed
by 3 digits (ex: P001).")
            continue
        else:
            break

```

```

try:
    with open('product.txt', 'r') as productfile:
        lines = productfile.readlines()
except IOError as e:
    print(e)
    print("Cannot read from file!")

product_found = False
updated_products = [] # create a list to store all the product information including the
updated product information
change_product = [] # create a list to temporary store current product information that
user wanted to update

for line in lines:
    if line.strip() == "" and change_product:
        for item in change_product:
            if item.startswith("Product ID:") and product_id_to_update in item:
                product_found = True
                print("\nCurrent product details:")
                print("".join(change_product)) # Display current product information

                #keep the original product information if user don't want to update
                original_name = change_product[1]
                original_description = change_product[2]
                original_price = change_product[3]
                original_quantity = change_product[4]

                #ask user if they want to change product ID
                while True:
                    update_product_id = input("Do you want to update the Product ID? (y/n): ")

                    if update_product_id == 'y':
                        while True:
                            new_product_id = input("Enter new product ID: ")

                            # Check if the product ID format is valid
                            if len(new_product_id) != 4 or not new_product_id.startswith('P') or not
new_product_id[1:].isdigit():
                                print("Invalid Product ID format. Please enter a valid product ID (ex:
P002).")
                                continue

                            # Check if the new product ID already exists in the file
                            id_exists = False
                            for line in lines:
                                if line.startswith("Product ID: ") and new_product_id in line:
                                    print(f"This Product ID {new_product_id} already exists. Please
enter a different one.")
                                    id_exists = True
                                    break # Exit the loop if the ID exists

                            if not id_exists:
                                change_product[0] = (f"Product ID: {new_product_id}\n")
                                break # Exit and break the inner while loop after successfully

```

updating the Product ID

```
break # Exit the outer while loop after completing the update process
```

```
elif update_product_id == 'n':  
    print("Product ID remains unchanged.")  
    break # Exit the outer loop if the user doesn't want to update the
```

Product ID

```
else:  
    print("Invalid input! Please enter 'y' for yes or 'n' for no.")
```

```
#ask user if they want to update product name
```

```
while True:
```

```
    product_name = input("Do you want to change product name? (y/n): ")
```

```
    if product_name == 'y':
```

```
        new_productname = input("Enter new product name: ")
```

```
        change_product[1] = (f"Product Name: {new_productname}\n") #
```

```
        break #break the inner while loop
```

```
    elif product_name == 'n':
```

```
        change_product[1] = original_name #keep the original product name if
```

user don't want to update it

```
        break #break the inner while loop, proceed to the next stage
```

```
    else:
```

```
        print("Invalid input! Please enter 'y' or 'n'.")
```

```
#ask user if they want to update product description
```

```
while True:
```

```
    product_description = input("Do you want to change product description?
```

(y/n): ")

```
    if product_description == 'y':
```

```
        new_productdescription = input("Enter new product description: ")
```

```
        change_product[2] = (f"Product Description:
```

{new_productdescription}\n")

```
        break
```

```
    elif product_description == 'n':
```

```
        change_product[2] = original_description #keep the original product
```

description if user don't want to update it

```
        break
```

```
    else:
```

```
        print("Invalid input! Please enter 'y' or 'n'.")
```

```
#ask user if they want to update product price
```

```
while True:
```

```
    product_price = input("Do you want to change product price? (y/n): ")
```

```
    if product_price == 'y':
```

```
        while True:
```

```
            try:
```

```
                new_productprice = float(input("Enter new product price: RM"))
```

```
                if new_productprice <= 0:
```

```
                    print("Product price cannot be negative! Please enter a valid
```

product price.")

```

        else:
            change_product[3] = (f"Product Price: RM{new_productprice}\n")
            break #if new product price user enter is valid, store new product
product into the change_product list
        except:
            print("Invalid input! Please enter a valid product price.")
            break
    elif product_price == 'n':
        change_product[3] = original_price #if user choose 'n' keep the original
product price if user don't want to update it
        break
    else:
        print("Invalid input! Please enter 'y' or 'n'.")

#ask user if they want to update product quantity
while True:
    product_quantity = input("Do you want to update product quantity? (y/n): ")
    if product_quantity == 'y':
        while True:
            try:
                new_productquantity = int(input("Enter new product quantity: "))
                if new_productquantity < 1:
                    print("Quantity must be a positive number.")
                else:
                    change_product[4] = (f"Product Quantity:
{new_productquantity}\n")
                    break
            except:
                print("Invalid input! Quantity must be an integer.")
                break
        elif product_quantity == 'n':
            change_product[4] = original_quantity #keep the original product
quantity if user don't want to update
        break
    else:
        print("Invalid input! Please enter 'y' or 'n'.")

print("Product updated successfully!")

    updated_products.extend(change_product) #add the updated product information
back to the final list
    updated_products.append("\n") # add blank line between products
    change_product = [] # reset the list
else:
    change_product.append(line)

if change_product:
    updated_products.extend(change_product)

if not product_found:
    print(f"Product ID {product_id_to_update} does not exist in the product file!") #print
this line then end, exit the function

```

```

# Write the updated product information back to the product text file
try:
    with open('product.txt', 'w') as productfile:
        productfile.writelines(updated_products)
except IOError as e:
    print(e)
    print("Error writing in file!")

```

Add a new supplier function

#a function that allow user to add new supplier information and store in the supplier.txt file

```
def add_supplier():
```

```

    #this loop will break if user enter a valid supplier id with no duplicate
    while True:
        supplierid = input("Enter supplier ID (ex: S001): ")
        if len(supplierid) != 4 or not supplierid.startswith('S') or not supplierid[1:].isdigit():
            print("Invalid Supplier ID format. Please enter supplier id with 'S' followed by 3 digits number.(ex: S001)")
            continue

        # Check if the supplier ID user enter already exists in the 'supplier.txt' file
        try:
            with open('supplier.txt', 'r') as supplierfile:
                lines = supplierfile.readlines()
                for line in lines:
                    if line.startswith("Supplier ID: ") and supplierid in line:
                        print("This Supplier ID already exists.") # Inform the user if the supplier ID
already exists

                        # Prompt user to decide whether to try again
                        while True:
                            user_choice = input("Do you want to enter supplier id again? (y/n): ")
                            if user_choice == 'y':
                                break # If user chooses 'y', prompt to enter product ID again
                            elif user_choice == 'n':
                                return # Exit the function if user chooses 'n'
                            else:
                                print("Please enter 'y' for yes or 'n' for no.") # If user enters invalid
input, ask them to enter 'y' or 'n' only
                                break
                        else:
                            #Break the loop if user enters a valid supplier ID format and no duplicate is
found
                            break
        except IOError as e:
            print(e)
            print("Cannot read from file!")

```

```

# Ask user to enter supplier name and contact number
supplier_name = input("Enter supplier name: ")
supplier_contact = input("Enter supplier contact number: ")

# Store the new supplier information into the 'supplier.txt' file
try:
    with open('supplier.txt', 'a') as supplierfile: # Store the supplier information user
entered into the supplier.txt file
        supplierfile.write(f"Supplier ID: {supplierid}\n")
        supplierfile.write(f"Supplier Name: {supplier_name}\n")
        supplierfile.write(f"Supplier Contact Number: {supplier_contact}\n")
        supplierfile.write("\n") # Add a blank line to separate entries
except IOError as e:
    print(e)
    print("Cannot read from file!")

print("Supplier added successfully!")

```

Place order function

```

#a function that allow user to place an order from the supplier
def place_order():
    # Order ID loop. This loop will break if user enter valid order id
    while True:
        orderid = input("Enter Order ID (ex: O001): ")

        # Check if the order ID format user entered is valid
        if len(orderid) != 4 or not orderid.startswith('O') or not orderid[1:].isdigit():
            print("Invalid Order ID format. Please enter Order ID starting with 'O' followed by 3
digits (ex: O001).")
            continue

        # Check if the order ID already exists in the order.txt file
        try:
            with open('order.txt', 'r') as orderfile:
                lines = orderfile.readlines()
                for line in lines:
                    if line.startswith("Order ID: ") and orderid in line:
                        print("This order ID already exists.")

            # Prompt user to decide whether to try again
            while True:
                user_choice = input("Do you want to enter order ID again? (y/n): ")
                if user_choice == 'y':
                    break # Exit while loop and ask for Order ID again
                elif user_choice == 'n':
                    return # Exit and return to main menu
                else:
                    print("Please enter 'y' for yes and 'n' for no.")

```

```

        break
    else:
        break

except IOError as e:
    print(e)
    print("Cannot read from file!")

# Order product ID loop. This loop will break if user enter valid product id to order
while True:
    productid_to_order = input("Enter Product ID: ")

    try: # Read the product file
        with open('product.txt', 'r') as productfile:
            products = productfile.readlines()
    except IOError as e:
        print("Error reading product file:", e)
        print("Cannot read from file!")
        return

    product_found = False
    product_data = [] # Create a list to store product data

    for line in products:
        if line.strip() == "" and product_data:
            for item in product_data:
                if item.startswith("Product ID:") and item.strip() == f"Product ID:
{productid_to_order}":
                    product_found = True
                    print("\nCurrent product details:")
                    print("".join(product_data)) # Display current product details
                    break

            if product_found:
                break # Exit if product found
            product_data = [] # Reset the list to gather details for the next product
        else:
            product_data.append(line)

    if not product_found:
        print(f"Product ID {productid_to_order} not found in inventory.")
        while True:
            user_enterchoice = input("Do you want to enter the Product ID again? (y/n): ")
            if user_enterchoice == 'y':
                break
            elif user_enterchoice == 'n':
                return # Exit the function
            else:
                print("Please enter 'y' or 'n'.")
        else:
            break # Exit the loop when a valid product is found

# Order quantity loop. This loop will break if user enter valid quantity for order

```

```

while True:
    try:
        order_quantity = int(input("Enter order quantity (for restock): "))
        if order_quantity < 1:
            print("Invalid input! Quantity must be a positive number.")
        else:
            break
    except ValueError:
        print("Error: Please enter a positive loop number for order quantity!")

# Prompt user to enter order date
order_date = input("Enter order date (DD-MM-YYYY): ")

# Supplier ID loop. This loop will check if user enter a valid supplier ID
while True:
    supplier_id = input("Enter Supplier ID: ")

    # Check if Supplier ID user entered already exists in the supplier file
    try:
        with open('supplier.txt', 'r') as supplierfile:
            lines = supplierfile.readlines()
            supplier_found = False

            for line in lines:
                if line.startswith("Supplier ID: ") and f"Supplier ID: {supplier_id}" == line.strip():
                    supplier_found = True
                    break # If found, exit the loop

            if not supplier_found:
                print(f"Supplier ID {supplier_id} not found in the inventory.")
                # Prompt user to decide whether to try again
                while True:
                    choice = input("Do you want to enter Supplier ID again? (y/n): ")
                    if choice == 'y':
                        break
                    elif choice == 'n':
                        return # Exit and function
                    else:
                        print("Please enter 'y' for yes and 'n' for no.")
            else:
                break # Exit the loop if a valid supplier ID is found

    except IOError as e:
        print("Error reading the supplier file:", e)
        return

# Prompt user to enter payment term
paymentterm = input("Enter payment term: ")

# Store the information user enter into the order.txt file
try:
    with open('order.txt', 'a') as orderfile:

```

```

orderfile.write(f"Order ID: {orderid}\n")
orderfile.write(f"Product ID: {productid_to_order}\n")
orderfile.write(f"Order Quantity (restock): {order_quantity}\n")
orderfile.write(f"Order Date: {order_date}\n")
orderfile.write(f"Supplier ID: {supplier_id}\n")
orderfile.write(f"Payment Term: {paymentterm}\n")
orderfile.write("\n")
print("\nSupplier order placed successfully!")
except IOError as e:
    print(f"An error occurred while appending to the file: {e}")

```

View Inventory function

```

# a function that allow user to view the inventory level of all products in that stored in the
inventory management system
def view_inventory():
    print("-"*64)
    print("-"*16,"Product Inventory Report","-"*22)
    print("-"*64)

# show how the inventory level in determined to user
print("Inventory level: ")
print("Low stock: Product Quantity <= 20")
print("Medium stock: Product Quantity between 20 to 100")
print("High stock: Product Quantity > 100")
print("-" * 64)

print(f"{'Product ID': <14}{'Product Name': <20} {'Quantity': <12} {'Inventory Level': <10}") #display product id, name, quantity, stock level left align with space
print("-" * 64)

#create a list that store product information
product_stock = []

try:
    with open('product.txt', 'r') as productfile:
        product_info = productfile.read().strip().split("\n\n") # Split by blank lines between
products

        for product in product_info:
            lines = product.split("\n")

            # Extract product information from product text file
            product_id = lines[0].split(":")[1].strip()
            product_name = lines[1].split(":")[1].strip()
            quantity = int(lines[4].split(":")[1].strip())

            # Determine stock level
            if quantity <= 20:

```

```

        stock_level = "Low Stock"
    elif quantity > 20 and quantity <= 100:
        stock_level = "Medium Stock"
    else:
        stock_level = "High Stock"

    #add the product information into the list
    product_stock.append((product_id, product_name, quantity, stock_level))

    #sort the product quantity from lowest to highest to view the inventory level
    #used key in lamba function to sort the quantity
    product_stock.sort(key=lambda x: x[2]) # Sort by the 3rd element (quantity)

    # Display sorted product details
    for product_id, product_name, quantity, stock_level in product_stock:
        print(f"{product_id:<15}{product_name:<22}{quantity:<12}{stock_level}")

except IOError as e:
    print(e)
    print("Cannot read from file!")

print("-" * 64)
print("[End of view inventory report]")

```

Generate report function

```

#a function that allow user to view the low stock items report, supplier information report,
supplier order reports and total product reports
def general_report():
    print("-" * 76)
    print("-" * 26, "General Report", "-" * 34)
    print("-" * 76)

    print("[Low Stock Items Report]")
    print("\nLow Stock Items (Quantity <= 20):")
    print(f"{'Product ID': <14}{'Product Name': <15}{'Product Quantity': <10}")
    print("-" * 76)

    low_stock = [] #store low stock items

    try:
        with open('product.txt', 'r') as productfile:
            product_info = productfile.read().strip().split("\n\n") # Split by blank lines between
products

            for product in product_info:
                lines = product.split("\n")

                # Extract product details
                product_id = lines[0].split(":")[1].strip()

```

```

        product_name = lines[1].split(":")[1].strip()
        quantity = int(lines[4].split(":")[1].strip())

        if quantity < 20:
            low_stock.append((product_id, product_name, quantity))

    low_stock.sort(key=lambda x: x[2])#sort the low stock product

    if low_stock:
        for product_id, product_name, quantity in low_stock:
            print(f'{product_id:<14}{product_name:<20}{quantity:<10}')
        else:
            print("No low stock items found.")
    except IOError:
        print("Cannot read from file!")

    print("-" * 76)

# Supplier report
print("[Supplier Report]")
print("[Supplier information]")
#print("\n")
print(f'{'Supplier ID': <13}{'Supplier Name': <16} {'Contact Number': <15}')
print("-" * 76)

supplier = [] #store supplier information

try:
    with open('supplier.txt', 'r') as supplierfile:
        suppliers = supplierfile.read().strip().split("\n\n")

        for sup in suppliers:
            lines = sup.split("\n")

            supplier_id = lines[0].split(":")[1].strip()
            supplier_name = lines[1].split(":")[1].strip()
            supplier_contact = lines[2].split(":")[1].strip()

            #add the supplier information into the list
            supplier.append((supplier_id, supplier_name, supplier_contact))

        if supplier:
            for supplier_id, supplier_name, supplier_contact in supplier:
                print(f'{'supplier_id:<14}{'supplier_name:<20}{'supplier_contact}')
            else:
                print("No supplier found.")
        except IOError:
            print("Cannot read from file!")

    print("-"*76)

# Supplier Order Report

```

```

print("[Supplier Order Report]")
print("\n[Supplier order summary]")
print(f'{"Supplier ID": <13}{"Reorder Product": <16} {"Order quantity": <15} {"Order date": <14} {"Payment Term": <10}')}
print("-" * 76)

supplier_order = [] #store supplier order summary information

try:
    with open('order.txt', 'r') as supplierfile:
        supplier_info = supplierfile.read().strip().split("\n\n")

        for order in supplier_info:
            lines = order.split("\n")
            product_id = lines[1].split(":")[1].strip()
            reorder_quantity = int(lines[2].split(":")[1].strip())
            order_date = lines[3].split(":")[1].strip()
            supplier_id = lines[4].split(":")[1].strip()
            payment_term = lines[5].split(":")[1].strip()

            # Add supplier order information into the supplier order list
            supplier_order.append((supplier_id, product_id,
reorder_quantity,order_date,payment_term))

            # Display the supplier orders information
            if supplier_order:
                for supplier_id, product_id, reorder_quantity,order_date,payment_term in
supplier_order:
                    print(f'{"supplier_id:<14}{"product_id:<20}{"reorder_quantity:<12} {"order_date:
<15}{"payment_term :<10}')}
            else:
                print("No supplier orders found.")

except IOError:
    print("Cannot read from file!")

print("-" * 76) # End of Supplier Order Report

#total products report, display the total product that store in this inventory management
system
print("[Product Report]")
print("\n[Total products in inventory]")
print(f'{"Product ID": <13}{"Product Name": <14} {"Quantity": <9} {"Unit Price(RM)": <14}
{'Total Product Price(RM)": <10}')}
print("-"*76)

product = [] # store product information
total_iventoryvalue = 0 #initialise the total inventory value first for the calculation later
try:
    with open('product.txt', 'r') as productfile:
        product_data = productfile.read().strip().split("\n\n") # Split by blank lines between
products

        for items in product_data:

```

```

lines = items.split("\n")

# Extract product details
product_id = lines[0].split(":")[1].strip()
product_name = lines[1].split(":")[1].strip()
product_price = lines[3].split(":")[1].strip()
product_quantity = int(lines[4].split(":")[1].strip())

# Convert product price to a float (removing RM)
product_price = float(product_price.replace("RM", "").strip())

# Calculate total product price
total_product_price = product_quantity * product_price

#calculate total inventory value
total_inventoryvalue += total_product_price

# add product details to the product list
product.append((product_id, product_name, product_quantity, product_price,
total_product_price))

# Display product information
if product:
    for product_id, product_name, product_quantity, product_price, total_stock_price in
product:
print(f'{product_id:<13}{product_name:<16}{product_quantity:<10}{product_price:<14.2f}{t
otal_stock_price:<20.2f}')

    print("-"*74)
    print(f'{Total Inventory Value in this inventory management system is:
RM}{total_inventoryvalue:.2f}')
    else:
        print("No product in inventory found.")

except IOError:
    print("Cannot read from file!")

print("-"*74)
print("[End of report]")

```

Main Program

#This main program combines all the functions in the inventory management system and allows the user to select an action to perform

```
def main_program():
```

```
    while True:
```

```

#display the main menu
print("-" * 64)
print("-" * 16, "Inventory Management System", "-" * 19)
print("-" * 64)
print("1. Add new product")
print("2. Update product details")
print("3. Add new supplier")
print("4. Place an order(for restock)")
print("5. View Inventory")
print("6. Generate Report")
print("7. Exit the program")
print("-" * 64)

try:
    choice = int(input("Please enter a choice: "))

    if choice == 1:
        add_product()
    elif choice == 2:
        update_product_details()
    elif choice == 3:
        add_supplier()
    elif choice == 4:
        place_order()
    elif choice == 5:
        view_inventory()
        break #will break and let user view and read the inventory report, do not loop
back to the main menu
    elif choice == 6:
        generate_report()
        break #will break and let user view and read the report
    elif choice == 7:
        print("Exit program. Thank you for using this Inventory Management System!")
        break # Break the loop and exit the program
    else:
        print("Invalid choice! Please enter a number between 1 to 7 for selection.")
except ValueError: #handle the error if users enter input that is not integer
    print("Invalid choice! Please enter a number between 1 to 7 for selection.")

# Call out the function
main_program()

```